

BobaLink Browser Extension

API Reference

Version: 1.0.0

Date: 2026-06-06

Classification: Public — Developer Reference

Table of Contents

1. Overview
2. Internal Extension APIs
3. Boba API Integration
4. qBitTorrent WebUI API Reference
5. TypeScript Type Definitions
6. Error Codes and Handling
7. Rate Limiting
8. Version Compatibility Matrix

1. Overview

This document provides exhaustive API reference material for three categories:

- 1. **Internal Extension APIs:** The message-passing protocol between content manager, popup, and options page.
- 2. **Boba API Integration:** RESTful endpoints exposed by the Boba orchestrator for authentication, search, download, and SSE streaming.
- 3. **qBitTorrent WebUI API:** Complete endpoint documentation for direct access to the qBitTorrent API, including all parameters and curl/JS fetch examples.

All code examples are provided in **curl**, **JavaScript (fetch)**, and **TypeScript**. Response schemas are specified in TypeScript interface notation.

Base URLs

Environment	Boba Base URL	qBitTorrent Base URL
Local (default)	<code>https://boba.local:8443</code>	<code>https://localhost:8080</code>
Docker	<code>https://boba:8443</code>	<code>https://boba:8080</code>
Custom	User-configured	User-configured

Authentication Summary

2. Internal Extension APIs

The extension uses `chrome.runtime.sendMessage()` for communication follow a uniform envelope format.

2.1 Message Envelope

```
interface ExtensionMessage<T = unknown> {  
    /** Message type discriminator */  
    type: MessageType;  
  
    /** Request correlation ID for async response matching */  
    requestId?: string;  
  
    /** Message payload – type depends on `type` */  
    payload: T;  
  
    /** Sender context (auto-populated by runtime) */  
    sender?: {  
        tabId?: number;  
        frameId?: number;  
        url?: string;  
    };  
}
```

```
type MessageType =  
    | 'SCAN_REQUEST'  
    | 'SCAN_RESPONSE'  
    | 'TORRENT_DETECTED'  
    | 'SEND_TORRENT'  
    | 'SEND_BATCH'  
    | 'GET_STATUS'  
    | 'STATUS_RESPONSE'  
    | 'GET_QUEUE'  
    | 'QUEUE_RESPONSE'  
    | 'RETRY_ITEM'  
    | 'REMOVE_ITEM'  
    | 'CLEAR_QUEUE'  
    | 'GET_CONFIG'  
    | 'CONFIG_RESPONSE'
```

Direction: Service Worker → Content Script

Delivery: `chrome.tabs.sendMessage(tabId, message)`

```
interface ScanRequestPayload {  
  /** Unique request identifier for correlation */  
  requestId: string;  
  
  /** When true, scan invisible/hidden links as well */  
  deepScan: boolean;  
  
  /** Optional CSS selector to scope the scan */  
  scope?: string;  
}
```

Example — Sending from Service Worker:

```
// TypeScript  
const requestId = crypto.randomUUID();  
const response = await chrome.tabs.sendMessage(tabId, {  
  type: 'SCAN_REQUEST',  
  requestId,  
  payload: { requestId, deepScan: false }  
});
```

```
// JavaScript  
const requestId = crypto.randomUUID();  
chrome.tabs.sendMessage(tabId, {  
  type: 'SCAN_REQUEST',  
  requestId,  
  payload: { requestId, deepScan: false }  
}, (response) => {  
  if (chrome.runtime.lastError) {
```

```

interface ScanResponsePayload {
  /** Correlation ID matching the SCAN_REQUEST */
  requestId: string;

  /** Detected torrents (deduplicated by content script) */
  torrents: TorrentInfo[];

  /** Error message if scan failed */
  error?: string;

  /** Metadata about the scan */
  meta: {
    linksScanned: number;
    magnetsFound: number;
    torrentFilesFound: number;
    scanDurationMs: number;
  };
}

```

2.4 TORRENT_DETECTED

Sent proactively by the content script when magnet links or torrent file links are detected while browsing (via `MutationObserver`).

Direction: Content Script → Service Worker

Delivery: `chrome.runtime.sendMessage(message)`

```

interface TorrentDetectedPayload {
  /** Array of newly detected torrents */
  torrents: TorrentInfo[];

  /** URL of the page where detection occurred */
}

```

```

const magnetLinks = document.querySelectorAll('a[href^="magne
const torrents = Array.from(magnetLinks).map(parseMagnetElement

if (torrents.length > 0) {
  chrome.runtime.sendMessage({
    type: 'TORRENT_DETECTED',
    payload: {
      torrents,
      pageUrl: location.href,
      timestamp: Date.now()
    }
  });
}

```

2.5 SEND_TORRENT

Queue or immediately send a single torrent to the configured download server

Direction: Popup/Context Menu → Service Worker

```

interface SendTorrentPayload {
  /** The torrent to send */
  torrent: TorrentInfo;

  /** Send immediately even if queue has items */
  priority?: 'normal' | 'high';

  /** Override default category for this send */
  category?: string;

  /** Override default tags for this send */
  tags?: string[];
}

```

2.6 SEND_BATCH

Send multiple torrents as a single batch operation.

Direction: Tab Manager → Service Worker

```
interface SendBatchPayload {  
  /** Array of torrents to send */  
  torrents: TorrentInfo[];  
  
  /** Optional category override for the entire batch */  
  category?: string;  
  
  /** Source tab group ID, for logging */  
  tabGroupId?: number;  
}
```

Response:

```
interface SendBatchResponse {  
  /** Number of torrents successfully queued/sent */  
  accepted: number;  
  
  /** Number of torrents rejected (e.g., duplicates) */  
  rejected: number;  
  
  /** Number of torrents that failed immediately */  
  failed: number;  
  
  /** Detailed results per torrent */  
  results: Array<{  
    infohash: string;  
    status: 'queued' | 'sent' | 'duplicate' | 'failed' |
```

```
// Request has empty payload
interface GetStatusPayload {}

interface StatusResponsePayload {
    /** Server connection state */
    connection: 'connected' | 'connecting' | 'disconnected' | '

    /** Server URL being used */
    serverUrl: string;

    /** Server version, if connected */
    serverVersion?: string;

    /** Number of items in the offline queue */
    queueSize: number;

    /** Number of items currently retrying */
    pendingCount: number;

    /** Number of items in dead-letter state */
    deadLetterCount: number;

    /** Last successful health check timestamp */
    lastHealthCheck?: number;

    /** Extension version */
    extensionVersion: string;
}
```

2.8 Queue Management Messages

GET_QUEUE

```
interface GetQueuePayload {
    /** Filter by status; omit for all */
    filter?: 'pending' | 'retrying' | 'failed' | 'dead-letter';

    /** Maximum items to return (default 100) */
    limit?: number;

    /** Offset for pagination */
    offset?: number;
}

interface QueueResponsePayload {
    items: QueueItem[];
    total: number;
    filtered: number;
}
```

RETRY_ITEM

```
interface RetryItemPayload {
    /** Queue item ID to retry */
    itemId: string;
}
```

REMOVE_ITEM

```
interface RemoveItemPayload {
    /** Queue item ID to remove */
    itemId: string;
}
```

2.9 Configuration Messages

GET_CONFIG

```
// Request: empty payload
// Response: full ExtensionConfig object (see Type Definition)
```

UPDATE_CONFIG

```
interface UpdateConfigPayload {
  /** Partial config update – only provided fields are merged
   * config: DeepPartial<ExtensionConfig>;
  }
}
```

The service worker validates all fields, applies the merge, and persists to encrypted storage. A `CONFIG_UPDATED` event is broadcast to all listening contexts.

2.10 Discovery Messages

DISCOVER_SERVERS

```
interface DiscoverServersPayload {
  /** Probe timeout in ms (default 5000) */
  timeout?: number;

  /** Specific IP addresses to probe; uses defaults if omitted
   * targets?: string[];
  }
}
```

DISCOVERY_RESPONSE

```
interface DiscoveryResponsePayload {  
    /** Discovered server endpoints */  
    servers: Array<{  
        url: string;  
        name: string;  
        version: string;  
        responseTimeMs: number;  
        healthy: boolean;  
    }>;  
  
    /** Number of targets probed */  
    targetsProbed: number;  
  
    /** Scan duration in ms */  
    durationMs: number;  
}
```

3. Boba API Integration

The Boba server acts as a proxy and orchestration layer between the extension and the Boba API, providing additional capabilities including search aggregation, metadata enrichment, and streaming.

3.1 Authentication

```
curl -X POST https://boba.local:8443/api/v1/auth/token \
  -H "Content-Type: application/json" \
  -d '{"apiKey": "bob_live_abc123xyz"}'
```

JavaScript (fetch):

```
const response = await fetch('https://boba.local:8443/api/v1/auth/token', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ apiKey: 'bob_live_abc123xyz' })
});
const data = await response.json();
// data.token – JWT for subsequent requests
// data.expiresAt – Unix timestamp of expiry
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5Ij06IiwiaXNjaWkiOiJkb2R5IiwiaWF0IjoxNzQ0MjY0MDA",
  "expiresAt": 1735689600,
  "type": "Bearer"
}
```

Response (401 Unauthorized):

```
{
  "error": "E_AUTH_INVALID_KEY",
  "message": "The provided API key is invalid or revoked."
}
```

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...new...",
  "expiresAt": 1735776000
}
```

3.2 Health Check

GET /api/v1/health

Check Boba server availability and version. No authentication required.

curl:

```
curl https://boba.local:8443/api/v1/health
```

JavaScript:

```
const response = await fetch('https://boba.local:8443/api/v1/health');
const data = await response.json();
```

Response (200 OK):

```
{
  "status": "healthy",
  "version": "1.4.2",
  "uptime": 86400,
  "extensions": {
    "qbittorrent": "connected",
    "search": "available",
  }
}
```

Parameter	Type	Required	Description
q	string	Yes	Search query string
category	string	No	Filter by category (e.g., movie)
limit	integer	No	Max results (default 50, max 100)
offset	integer	No	Pagination offset

curl:

```
curl "https://boba.local:8443/api/v1/torrents/search?q=inception" \
  -H "X-API-Key: bob_live_abc123xyz"
```

JavaScript:

```
const params = new URLSearchParams({
  q: 'inception 2010',
  category: 'movies',
  limit: '10'
});
const response = await fetch(
  `https://boba.local:8443/api/v1/torrents/search?${params}`,
  { headers: { 'X-API-Key': 'bob_live_abc123xyz' } }
);
const data = await response.json();
```

Response (200 OK):

```
{
  "results": [
    {
      "infohash": "a1b2c3d4e5f6789012345678901234567890abcd",
      "displayName": "Inception (2010) 1080p BluRay",
      "magnetUri": "magnet:?xt=urn:btih:a1b2c3d4e5f6789012345678901234567890abcd",
      "size": 2147483648,
      "seeders": 150,
      "leechers": 25,
      "source": "1337x",
      "category": "movies",
      "uploadedAt": "2026-01-15T08:30:00Z"
    }
  ],
  "total": 47,
  "query": "inception 2010",
  "categories": ["movies", "tv"]
}
```

3.4 Download API

POST /api/v1/torrents/download

Submit a torrent for download via Boba.

Request Body:

```
interface DownloadRequest {
    /** Magnet URI or infohash */
    magnetUri?: string;
    infohash?: string;

    /** Category for qBitTorrent */
    category?: string;

    /** Tags for qBitTorrent */
    tags?: string[];

    /** Override download path */
    savePath?: string;

    /** Rename the torrent */
    rename?: string;

    /** Pause after adding */
    paused?: boolean;

    /** Skip hash check */
    skipChecking?: boolean;

    /** Sequential download */
    sequentialDownload?: boolean;

    /** First/last piece priority */
    firstLastPiecePrio?: boolean;
}
```

curl:

```
curl -X POST https://boba.local:8443/api/v1/torrents/download
-H "Content-Type: application/json" \
```

```
const response = await fetch('https://boba.local:8443/api/v1/
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-API-Key': 'bob_live_abc123xyz'
  },
  body: JSON.stringify({
    infohash: 'a1b2c3d4e5f6789012345678901234567890abcd',
    category: 'movies',
    tags: ['1080p', 'blu-ray']
  })
});
const data = await response.json();
```

Response (202 Accepted):

```
{
  "jobId": "job_abc123",
  "status": "queued",
  "infohash": "a1b2c3d4e5f6789012345678901234567890abcd",
  "estimatedStart": "2026-06-06T12:05:00Z"
}
```

Response (409 Conflict — duplicate):

```
{
  "error": "E_DUPLICATE",
  "message": "Torrent already exists in download queue",
  "existingInfohash": "a1b2c3d4e5f6789012345678901234567890ab
}
```

JavaScript (EventSource):

```
const evtSource = new EventSource(
  'https://boba.local:8443/api/v1/events/download-progress?ap
);

evtSource.addEventListener('progress', (event) => {
  const data = JSON.parse(event.data);
  console.log(`Torrent ${data.infohash}: ${data.progress * 100}%`);
  console.log(`Speed: ${formatBytes(data.speed)}/s, ETA: ${data.eta}`);
});

evtSource.addEventListener('completed', (event) => {
  const data = JSON.parse(event.data);
  console.log(`Torrent completed: ${data.infohash}`);
});

evtSource.addEventListener('error', (event) => {
  console.error('SSE connection error');
  // Auto-reconnect handled by EventSource
});
```

Event Types:

Event	Data Fields
progress	infohash, progress, speed, eta, downloaded, total
completed	infohash, completionTime, ratio
error	infohash, error, code

```
interface ProgressEvent {
  infohash: string;
  progress: number;      // 0.0 to 1.0
  speed: number;         // Bytes per second
  eta: number;           // Estimated seconds remaining
  downloaded: number;    // Bytes downloaded
  total: number;         // Total bytes
  numSeeds: number;
  numLeechs: number;
  state: 'downloading' | 'stalledDL' | 'checkingDL' | 'metaDL'
}
```

4. qBitTorrent WebUI API Reference

This section documents the qBitTorrent WebUI API endpoints used by BobaL to the configured qBitTorrent base URL (e.g., `https://localhost:8080`).

4.1 Authentication

POST /api/v2/auth/login

Authenticate and establish a session.

Request:

Parameter	Type	Required	Des
-----------	------	----------	-----

JavaScript:

```
const params = new URLSearchParams();
params.append('username', 'admin');
params.append('password', 'adminadmin');

const response = await fetch('https://localhost:8080/api/v2/auth/login', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
    'Referer': 'https://localhost:8080'
  },
  body: params,
  credentials: 'include' // Important: accept and send cookies
});

if (response.status === 200) {
  const body = await response.text();
  // body === "Ok.SID" on success
  console.log('Login successful, SID cookie set');
} else {
  console.error('Login failed');
}
```

Response:

- **200 OK** with body `Ok.SID` — Success, `SID` cookie set.
- **403 Forbidden** — Invalid credentials.
- **429 Too Many Requests** — Too many failed login attempts.

Important Notes:

- The `Referer` header MUST match the qBitTorrent origin or the request will be rejected.
- The `SID` cookie is automatically managed by the browser's cookie store.
- Call `/api/v2/auth/logout` to terminate the session.

4.2 Torrent Management

POST /api/v2/torrents/add

Add one or more torrents (magnet URIs or `.torrent` files).

Request Parameters:

Parameter	Type	Required	Description
<code>urls</code>	string	No*	Magnet URL separated
<code>torrents</code>	file	No*	<code>.torrent</code>
<code>savepath</code>	string	No	Download d
<code>cookie</code>	string	No	Cookie to se
<code>category</code>	string	No	Category to
<code>tags</code>	string	No	Comma-sep
<code>skip_checking</code>	string (<code>true</code> / <code>false</code>)	No	Skip hash ch
<code>paused</code>	string (<code>true</code> / <code>false</code>)	No	Add paused
<code>root_folder</code>	string (<code>true</code> / <code>false</code>)	No	Create root f
<code>rename</code>	string	No	Rename torr
<code>upLimit</code>	integer	No	Upload limi
<code>dlLimit</code>	integer	No	Download li
<code>ratioLimit</code>	float	No	Share ratio l

curl — Magnet URI:

```
curl -X POST "https://localhost:8080/api/v2/torrents/add" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -F "urls=magnet:?xt=urn:btih:a1b2c3d4e5f6789012345678901234567890ab" \
  -F "category=movies" \
  -F "tags=hd,2026" \
  -F "paused=false"
```

curl — .torrent File:

```
curl -X POST "https://localhost:8080/api/v2/torrents/add" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -F "torrents=@/path/to/file.torrent" \
  -F "category=software" \
  -F "sequentialDownload=true"
```

JavaScript — Magnet URI:

```
const formData = new FormData();
formData.append('urls',
  'magnet:?xt=urn:btih:a1b2c3d4e5f6789012345678901234567890ab'
);
formData.append('category', 'movies');
formData.append('tags', 'hd,2026');

const response = await fetch('https://localhost:8080/api/v2/torrents/add', {
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: formData,
});
```

```

const fileInput = document.getElementById('torrent-file');
const file = fileInput.files[0];

const formData = new FormData();
formData.append('torrents', file, file.name);
formData.append('category', 'software');

const response = await fetch('https://localhost:8080/api/v2/torrents', {
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: formData,
  credentials: 'include'
});

```

Response Codes:

Status	Meaning
200	Success — torrent(s) added
400	Bad Request — missing urls/torrents
403	Not authenticated
415	Unsupported Media Type
500	Internal Server Error

GET /api/v2/torrents/info

List all torrents with optional filtering.

Query Parameters:

Parameter	Type	Description
<code>filter</code>	string	<code>all</code> , <code>downloading</code> , <code>seeding</code> , <code>completed</code> , <code>paused</code> , <code>resumed</code> , <code>stalled</code> , <code>stalled_uploading</code> , <code>stalled_downloading</code> , <code>errored</code>
<code>category</code>	string	Filter by category
<code>tag</code>	string	Filter by tag
<code>sort</code>	string	Sort column (e.g., <code>name</code> , <code>progress</code> , <code>added_on</code>)
<code>reverse</code>	boolean	Reverse sort order
<code>limit</code>	integer	Max results
<code>offset</code>	integer	Pagination offset
<code>hashes</code>	string	Pipe-separated infohashes

curl:

```
# All downloading torrents
curl "https://localhost:8080/api/v2/torrents/info?filter=downloading" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"

# Specific torrent by hash
curl "https://localhost:8080/api/v2/torrents/info?hashes=a1b2c3d4e5f6" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

```
const params = new URLSearchParams({
  filter: 'downloading',
  sort: 'progress',
  reverse: 'true',
  limit: '50'
});

const response = await fetch(
  `https://localhost:8080/api/v2/torrents/info?${params}`,
  {
    headers: { 'Referer': 'https://localhost:8080' },
    credentials: 'include'
  }
);
const torrents = await response.json();

for (const t of torrents) {
  console.log(`${t.name}: ${(t.progress * 100).toFixed(1)}%`);
  console.log(`  Speed: DL ${t.dlspeed} / UL ${t.upspeed}`);
  console.log(`  State: ${t.state}, ETA: ${t.eta}`);
}
```

Response (Array of Torrent objects):

```
[
  {
    "hash": "a1b2c3d4e5f6789012345678901234567890abcd",
    "name": "Ubuntu 24.04 LTS Desktop",
    "magnet_uri": "magnet:?xt=urn:btih:a1b2c3d4...",
    "size": 6270988288,
    "progress": 0.7534,
    "dlspeed": 5242880,
    "upspeed": 1048576,
    "priority": 1,
    "num_seeds": 150,
    "num_complete": 2500,
    "num_leechs": 25,
    "num_incomplete": 50,
    "ratio": 0.125,
    "eta": 1800,
    "state": "downloading",
    "seq_dl": false,
    "f_l_piece_prio": false,
    "category": "linux",
    "tags": "iso,official",
    "super_seeding": false,
    "force_start": false,
    "save_path": "/downloads/linux",
    "added_on": 1717776000,
    "completion_on": 0,
    "tracker": "https://torrent.ubuntu.com/announce",
    "dl_limit": 0,
    "up_limit": 0,
    "downloaded": 4724464025,
    "uploaded": 590558003,
    "downloaded_session": 1048576000,
    "uploaded_session": 209715200,
    "amount_left": 1546524263,
    "completed": 0,
    "max_ratio": -1,
```

POST /api/v2/torrents/delete

Delete one or more torrents.

Parameters:

Parameter	Type	Required	Description
hashes	string	Yes	Pipe-separated list of hashes
deleteFiles	string (true / false)	No	Also delete files

curl:

```
curl -X POST "https://localhost:8080/api/v2/torrents/delete" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "hashes=a1b2c3d4e5f6789012345678901234567890abcd" \
  -d "deleteFiles=false"
```

JavaScript:

```
const params = new URLSearchParams();
params.append('hashes', 'a1b2c3d4e5f6789012345678901234567890');
params.append('deleteFiles', 'false');

await fetch('https://localhost:8080/api/v2/torrents/delete', {
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: params,
  credentials: 'include'
});
```

Parameter	Type	Required	Description
hashes	string	Yes	Pipe-separated infohash

curl:

```
curl -X POST "https://localhost:8080/api/v2/torrents/pause" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "hashes=a1b2c3d4e5f6789012345678901234567890abcd"
```

JavaScript:

```
await fetch('https://localhost:8080/api/v2/torrents/pause', {
  method: 'POST',
  headers: { 'Referer': 'https://localhost:8080' },
  body: new URLSearchParams({
    hashes: 'a1b2c3d4e5f6789012345678901234567890abcd'
  }),
  credentials: 'include'
});
```

POST /api/v2/torrents/start

Resume paused torrent(s).

Parameters:

Parameter	Type	Required	Description
-----------	------	----------	-------------

Parameters:

Parameter	Type	Required	Description
hashes	string	Yes	Pipe-separated

curl:

```
curl -X POST "https://localhost:8080/api/v2/torrents/recheck"  
-b cookies.txt \  
-H "Referer: https://localhost:8080" \  
-d "hashes=a1b2c3d4e5f6789012345678901234567890abcd"
```

POST /api/v2/torrents/reannounce

Force reannounce to tracker(s).

Parameters:

Parameter	Type	Required	Description
hashes	string	Yes	Pipe-separated

4.3 Categories and Tags

GET /api/v2/torrents/categories

List all categories.

```
const response = await fetch(
  'https://localhost:8080/api/v2/torrents/categories',
  {
    headers: { 'Referer': 'https://localhost:8080' },
    credentials: 'include'
  }
);
const categories = await response.json();
// Returns: { "movies": { "name": "movies", "savePath": "/down
```

POST /api/v2/torrents/createCategory

Create a new category.

Parameter	Type	Required	Description
category	string	Yes	Category name
savePath	string	No	Default save path

curl:

```
curl -X POST "https://localhost:8080/api/v2/torrents/createCategory" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "category=4k-movies" \
  -d "savePath=/downloads/movies/4k"
```

GET /api/v2/torrents/tags

Parameter	Type	Required	Description
hashes	string	Yes	Pipe-separated infohash
tags	string	Yes	Comma-separated tags

curl:

```
curl -X POST "https://localhost:8080/api/v2/torrents/addTags" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d "hashes=a1b2c3d4e5f6789012345678901234567890abcd" \
  -d "tags=important,seeding"
```

4.4 Synchronization

GET /api/v2/sync/maindata

Get incremental sync data (all torrents, categories, tags, server state).

Parameter	Type	Required	Description
rid	integer	Yes	Response ID; use 0 for initial sync,

curl:

```
# Initial sync
curl "https://localhost:8080/api/v2/sync/maindata?rid=0" \
```

```

class QBitTorrentSync {
  private rid = 0;
  private intervalId: number | null = null;

  startPolling(intervalMs = 2000) {
    this.intervalId = window.setInterval(async () => {
      const response = await fetch(
        `https://localhost:8080/api/v2/sync/maindata?rid=${this.rid}`
      );
      const data = await response.json();
      this.rid = data.rid;
      this.handleUpdate(data);
    }, intervalMs);
  }

  private handleUpdate(data: MainDataResponse) {
    if (data.torrents) {
      // Full torrent data (initial sync)
      console.log('Full sync:', Object.keys(data.torrents).length);
    }
    if (data.torrents_removed) {
      console.log('Removed:', data.torrents_removed);
    }
    if (data.categories) {
      console.log('Categories updated:', data.categories);
    }
    if (data.server_state) {
      console.log('Global speeds:',
        `DL ${data.server_state.dl_info_speed}`,
        `UL ${data.server_state.up_info_speed}`
      );
    }
  }
}

```

```
{
  "rid": 43,
  "full_update": false,
  "torrents": {
    "a1b2c3d4...": {
      "progress": 0.7534,
      "dlspeed": 5242880,
      "upspeed": 1048576,
      "eta": 1800,
      "state": "downloading"
    }
  },
  "torrents_removed": [],
  "categories": {},
  "tags": [],
  "trackers": {},
  "server_state": {
    "connection_status": "connected",
    "dht_nodes": 350,
    "dl_info_speed": 5242880,
    "dl_info_data": 10737418240,
    "up_info_speed": 1048576,
    "up_info_data": 2147483648,
    "queueing": true,
    "use_alt_speed_limits": false,
    "refresh_interval": 2000
  }
}
```

4.5 Application

GET /api/v2/app/version

```
const response = await fetch('https://localhost:8080/api/v2/a
  headers: { 'Referer': 'https://localhost:8080' },
  credentials: 'include'
});
const version = await response.text();
console.log('qBitTorrent version:', version);
```

GET /api/v2/app/webapiVersion

Get WebUI API version.

curl:

```
curl "https://localhost:8080/api/v2/app/webapiVersion" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

Response: "2.9.3" (plain text)

GET /api/v2/app/preferences

Get application preferences.

curl:

```
curl "https://localhost:8080/api/v2/app/preferences" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

Response: Full preferences object (100+ fields).

```
curl -X POST "https://localhost:8080/api/v2/app/setPreference" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080" \
  -d 'json={"max_active_downloads": 5, "max_active_torrents": 5}'
```

GET /api/v2/app/buildInfo

Get build information.

curl:

```
curl "https://localhost:8080/api/v2/app/buildInfo" \
  -b cookies.txt \
  -H "Referer: https://localhost:8080"
```

Response:

```
{
  "qt": "6.6.2",
  "libtorrent": "2.0.10.0",
  "boost": "1.84.0",
  "openssl": "3.2.1",
  "zlib": "1.3.1"
}
```

4.6 Transfer Info

GET /api/v2/transfer/info

```
const response = await fetch('https://localhost:8080/api/v2/t
  headers: { 'Referer': 'https://localhost:8080' },
  credentials: 'include'
});
const info = await response.json();
console.log(`Global DL: ${formatBytes(info.dl_info_speed)}/s`);
console.log(`Global UL: ${formatBytes(info.up_info_speed)}/s`);
console.log(`DHT Nodes: ${info.dht_nodes}`);
```

Response:

```
{
  "connection_status": "connected",
  "dht_nodes": 350,
  "dl_info_speed": 5242880,
  "dl_info_data": 10737418240,
  "up_info_speed": 1048576,
  "up_info_data": 2147483648,
  "dl_rate_limit": 0,
  "up_rate_limit": 0,
  "queueing": true,
  "use_alt_speed_limits": false,
  "refresh_interval": 2000
}
```

4.7 qBitTorrent API Endpoint Summary

Method	Endpoint
POST	/api/v2/auth/login
POST	/api/v2/auth/logout
POST	/api/v2/torrents/add
GET	/api/v2/torrents/info
POST	/api/v2/torrents/delete
POST	/api/v2/torrents/pause
POST	/api/v2/torrents/start
POST	/api/v2/torrents/recheck
POST	/api/v2/torrents/reannounce
GET	/api/v2/torrents/categories
POST	/api/v2/torrents/createCategory
GET	/api/v2/torrents/tags
POST	/api/v2/torrents/addTags
POST	/api/v2/torrents/removeTags
GET	/api/v2/sync/maindata

5. TypeScript Type Definitions

5.1 Core Types

```
/**
 * Uniquely identifies a torrent by its 40-character SHA-1 in
 */
type Infohash = string; // /^[a-f0-9]{40}$/

/**
 * Magnet URI as defined by BEP-0009.
 */
type MagnetUri = string; // /^magnet:\?xt=urn:btih:[a-fA-F0-9]

/**
 * Extension-defined priority levels for queue management.
 */
type Priority = 'low' | 'normal' | 'high' | 'critical';

/**
 * Server connection states.
 */
type ConnectionState = 'connected' | 'connecting' | 'disconne

/**
 * Supported authentication methods.
 */
type AuthMethod = 'cookie' | 'api-key' | 'basic' | 'custom-he

/**
 * Supported connection modes.
 */
type ConnectionMode = 'boba' | 'qbittorrent-direct';

/**
 * UI theme preference.
 */
```

5.2 Data Interfaces

```

/**
 * Represents a detected torrent with all parsed metadata.
 */
interface TorrentInfo {
    infohash: Infohash;
    displayName: string;
    magnetUri: MagnetUri;
    source: TorrentSource;
    trackers: string[];
    totalSize?: number;
    webSeeds?: string[];
    pageUrl: string;
    detectedAt: number;
    fileBlob?: Blob;
}

/**
 * Encrypted credential storage entry.
 */
interface EncryptedCredentials {
    /** Base64-encoded salt (16 bytes) */
    salt: string;

    /** Base64-encoded IV (12 bytes) + ciphertext + authTag (16 bytes) */
    ciphertext: string;

    /** Timestamp of encryption */
    encryptedAt: number;
}

/**
 * Server connection configuration.
 */
interface ServerConfig {
    mode: ConnectionMode;
    baseUrl: string;
}

```

5.3 Queue Types

```
interface QueueItem {
  id: string;
  torrent: TorrentInfo;
  status: QueueStatus;
  attemptCount: number;
  nextRetryAt: number;
  lastError?: string;
  lastHttpStatus?: number;
  queuedAt: number;
}

interface QueueSnapshot {
  items: QueueItem[];
  total: number;
  byStatus: Record<QueueStatus, number>;
}
```

5.4 API Response Types

```
interface ApiResponse<T> {
    success: boolean;
    data?: T;
    error?: ApiError;
    meta?: {
        requestId: string;
        timestamp: number;
        durationMs: number;
    };
}

interface ApiError {
    code: string;
    message: string;
    details?: Record<string, unknown>;
    httpStatus?: number;
}

interface ConnectionTestResult {
    success: boolean;
    serverUrl: string;
    mode: ConnectionMode;
    version?: string;
    latencyMs: number;
    error?: string;
}
```

5.5 qBitTorrent-Specific Types

```
interface QBitTorrentInfo {
    hash: string;
    name: string;
    magnet_uri: string;
    size: number;
    progress: number;
    dlspeed: number;
    upspeed: number;
    priority: number;
    num_seeds: number;
    num_complete: number;
    num_leechs: number;
    num_incomplete: number;
    ratio: number;
    eta: number;
    state: TorrentState;
    seq_dl: boolean;
    f_l_piece_prio: boolean;
    category: string;
    tags: string;
    super_seeding: boolean;
    force_start: boolean;
    save_path: string;
    added_on: number;
    completion_on: number;
    tracker: string;
    dl_limit: number;
    up_limit: number;
    downloaded: number;
    uploaded: number;
    downloaded_session: number;
    uploaded_session: number;
    amount_left: number;
    completed: number;
    max_ratio: number;
    max_seeding_time: number;
```

5.6 Utility Types

```
/**
 * Deep partial type for partial config updates.
 */
type DeepPartial<T> = {
  [P in keyof T]?: T[P] extends object ? DeepPartial<T[P]> :
};

/**
 * Message handler function type.
 */
type MessageHandler<TPayload = unknown, TResponse = unknown> = (
  message: ExtensionMessage<TPayload>,
  sender: chrome.runtime.MessageSender
) => Promise<TResponse> | TResponse;

/**
 * Rate limit bucket state.
 */
interface RateLimitBucket {
  tokens: number;
  lastRefill: number;
  capacity: number;
}
```

6. Error Codes and Handling

6.1 Extension Error Codes

Code	HTTP Status	Description
E_UNKNOWN	—	Unexpected error
E_NETWORK	—	Network unreachable
E_TIMEOUT	408	Request timed out
E_DNS	—	DNS resolution failed
E_CONN_REFUSED	—	Connection refused
E_AUTH	401	Authentication failed
E_AUTH_EXPIRED	401	Session expired
E_RATE_LIMIT	429	Rate limited by server
E_CLIENT_RATE_LIMIT	—	Client rate limit hit
E_VALIDATION	400	Invalid request data
E_SERVER	500	Server error
E_SERVICE_UNAVAILABLE	503	Server temporarily unavailable
E_TORRENT_INVALID	400	Invalid torrent data
E_TORRENT_DUPLICATE	409	Torrent already exists
E_STORAGE_FULL	—	Local storage quota exceeded

6.2 Error Response Format

All API errors from Boba follow this format:

```
{
  "error": "E_RATE_LIMIT",
  "message": "Too many requests. Please slow down.",
  "retryAfter": 30,
  "details": {
    "limit": 60,
    "window": 60,
    "remaining": 0
  }
}
```

6.3 Error Handling in Extension Code

```
async function handleApiCall<T>(
  operation: () => Promise<T>,
  context: string
): Promise<T> {
  try {
    return await operation();
  } catch (error) {
    if (error instanceof ExtensionError) {
      switch (error.code) {
        case 'E_AUTH':
        case 'E_AUTH_EXPIRED':
          await promptReauthentication();
          throw error;

        case 'E_RATE_LIMIT':
          const delay = error.retryAfter || 60;
          await scheduleRetry(delay * 1000);
          throw error;

        case 'E_NETWORK':
        case 'E_TIMEOUT':
          await queueForRetry(context);
          throw error;

        case 'E_TORRENT_DUPLICATE':
          // Silently skip duplicates
          return null as T;

        default:
          await logError(error, context);
          throw error;
      }
    }
  }
}
```

6.4 Retry Strategies

Error Category	Strategy	Max Retries	B
Network errors	Exponential backoff	5	5
Timeout	Exponential backoff	3	2
Rate limit (429)	Honor <code>Retry-After</code>	10	A
Auth errors	No automatic retry	0	N
Server errors (5xx)	Exponential backoff with jitter	5	5
Validation errors	No retry	0	N

7. Rate Limiting

7.1 Client-Side Rate Limiter

The extension implements a token bucket rate limiter to prevent overwhelming

```

class TokenBucketRateLimiter {
  private tokens: number;
  private lastRefill: number;

  constructor(
    private capacity: number,
    private refillRate: number, // tokens per second
    private initialTokens: number = capacity
  ) {
    this.tokens = initialTokens;
    this.lastRefill = Date.now();
  }

  /**
   * Attempt to consume tokens. Returns wait time in ms if no
   */
  tryConsume(count: number = 1): { allowed: boolean; waitMs?: number } {
    this.refill();

    if (this.tokens >= count) {
      this.tokens -= count;
      return { allowed: true };
    }

    const deficit = count - this.tokens;
    const waitMs = Math.ceil((deficit / this.refillRate) * 1000);
    return { allowed: false, waitMs };
  }

  private refill(): void {
    const now = Date.now();
    const elapsedMs = now - this.lastRefill;
    const tokensToAdd = (elapsedMs / 1000) * this.refillRate;
    this.tokens = Math.min(this.capacity, this.tokens + tokensToAdd);
    this.lastRefill = now;
  }
}

```

7.3 Rate Limit Headers

The extension respects server-sent rate limit headers:

Header	Description
X-RateLimit-Limit	Request quota
X-RateLimit-Remaining	Remaining requests
X-RateLimit-Reset	Reset timestamp
Retry-After	Seconds to wait

7.4 Queue Processing Rate Limit

When processing the offline queue:

```
const QUEUE_CONFIG = {
  maxConcurrent: 1,
  interCallDelayMs: 500,
  batchSize: 10,
  batchIntervalMs: 5000
};

async function processQueue(items: QueueItem[]): Promise<void> {
  for (const item of items) {
    await sendTorrent(item.torrent);
    await delay(QUEUE_CONFIG.interCallDelayMs);
  }
}
```

8.1 qBitTorrent Version Compatibility

qBitTorrent	WebAPI	Status	Notes
4.4.x	2.8.x	Compatible	Legacy support
4.5.x	2.8.x	Compatible	Recommended
4.6.x	2.9.x	Compatible	Primary target
5.0.x	2.10.x	Compatible	Future-proof
5.1.x	2.11.x	Testing	Under validation
< 4.4.0	< 2.8.0	Unsupported	Upgrade required

8.2 Browser Version Compatibility

Browser	Minimum	Recommended	MV3 Support
Chrome	88	120+	Full
Firefox	109	121+	Full (with patches)
Opera	74	106+	Full
Yandex	21	24+	Full
Edge	88	120+	Full

8.3 Boba Server Compatibility

Boba Server	Extension	Status
1.0.x	1.0.x	Compatible
1.1.x	1.0.x	Compatible (back)
1.2.x	1.0.x	Testing
2.0.x	1.1.x+	Planned

8.4 API Feature Availability

Feature	qBitTorrent Direct	Boba Proxy
Magnet send	4.4+	All
.torrent upload	4.4+	All
Categories	4.4+	All
Tags	4.3.2+	All
Sequential download	4.4+	All
First/last piece prio	4.4+	All
SSE progress	N/A	1.1+